Assignment 1 – Attack Report

Md Nasif Sarwar

1. Reflected XSS

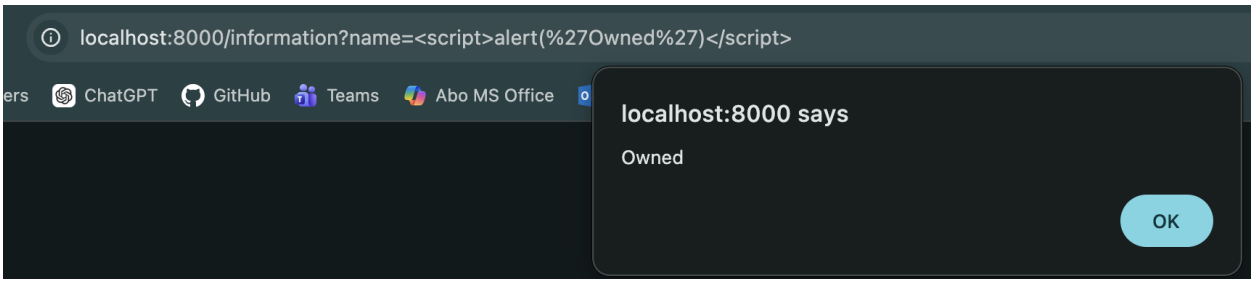
Cross-Site Scripting (XSS) occurs when an application includes untrusted input in a web page response without proper encoding. This allows an attacker to insert code (typically JavaScript) that executes in the victim's browser, within the framework of the vulnerable site.

The /information endpoint of the application reflects the query parameters directly into the rendered HTML template without escaping, making it vulnerable to reflected XSS.

1.1 The attack performed and proof of attack

The following URL was submitted to the application:

http://localhost:8000/information?name=<script>alert('Owned')</script>



When I visited the URL a JavaScript alert box popped up with the message "Owned". This proved that the injected script was executed by the browser and not displayed as plain text.

Looking at the page source (not the DOM returned by the browser, but the raw HTML returned by the server) confirms the cause . The /information handler injects the " name " query parameter directly into the HTML template, with no escaping.

```
<div class="container">
    <div class="jumbotron">
        <div class="container">
            <h1 class="display-4">My first web application</h1>

            <p>Request data</p>

            <ul>
                <li>Method : GET</li>
                <li>Path : /information</li>
                <li>Query string : ?name=%3Cscript%3Ealert(%27Owned%27)%3C/script%3E</li>
            </ul>

            <p>Parsed queries</p>

            <ul>
                <li>name = <script>alert('Owned')</script></li>
            </ul>

        </div>
    </div>
</div>
```

The server-side log for the request shows the raw, percent-encoded query string as received:

GET /information?name=%3Cscript%3Ealert(%27Owned%27)%3C/script%3E 1.1

```
Body:
GET /information?name=%3Cscript%3Ealert(%27Owned%27)%3C/script%3E 1.1
Content-Type: undefined
```

Here is an example of a reflected XSS vulnerability: the value of the "name" query parameter is reflected back into the response HTML directly, without any HTML encoding, so that an attacker can inject and execute arbitrary JavaScript within the victim's browser session.

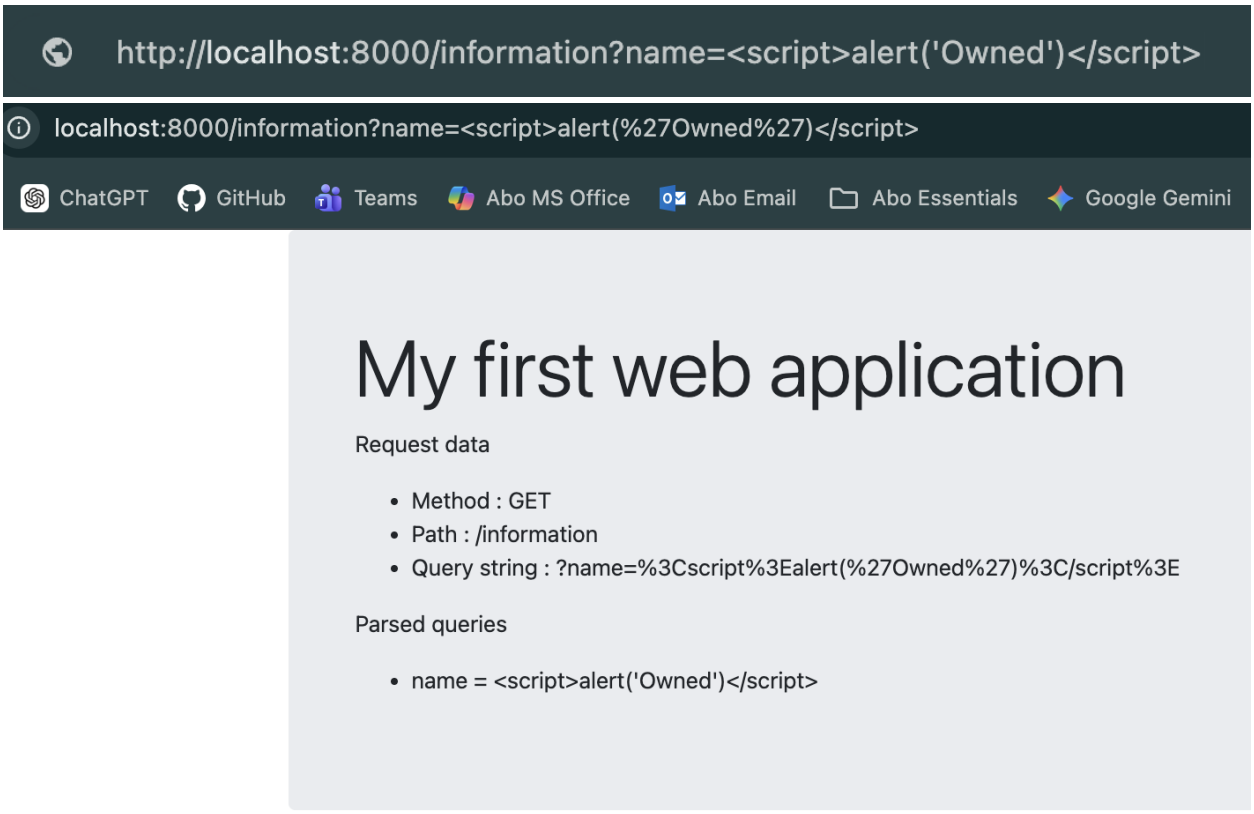
1.2 The countermeasures taken

Added an escapeHtml() helper function that replaces the HTML special characters &, <, >, " and ' with their corresponding HTML entities (&, <, >, ", '). This function is called for each piece of data derived from the request before it is inserted into the information.template output: the request method, the path, the raw query string, and each key and value of the parsed query parameters.

```
33  function escapeHtml(str) {
34    return String(str)
35      .replace(/&/g, "&amp;")
36      .replace(/</g, "&lt;")
37      .replace(/>/g, "&gt;")
38      .replace(/"/g, "&quot;")
39      .replace(/'/g, "&#39;");
40  }
41
42  function information(req, res, parsedUrl) {
43    var templatePath = path.join(__dirname, "templates", "information.template");
44
45    fs.readFile(templatePath, "utf8", (err, template) => {
46      var queryRows = Object.keys(parsedUrl.query)
47        .map(key => `<li>${escapeHtml(key)} = ${escapeHtml(parsedUrl.query[key])}</li>`)
48        .join("");
49
50      var html = template
51        .replace("{{method}}", escapeHtml(req.method))
52        .replace("{{path}}", escapeHtml(parsedUrl.pathname))
53        .replace("{{query}}", escapeHtml(parsedUrl.search || ""))
54        .replace("{{queries}}", queryRows);
55
56      res.writeHead(200, { "Content-Type": "text/html" });
57      res.end(html);
58    });
59  }
```

Re-visiting the same attack URL after the patch:

http://localhost:8000/information?name=<script>alert('Owned')</script>



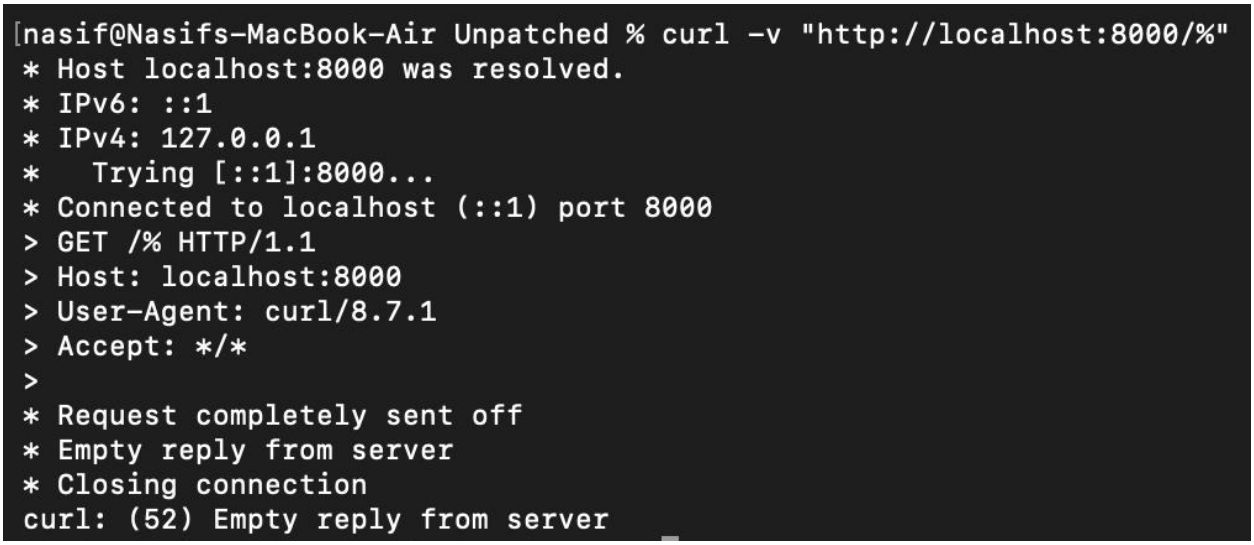
Because the special characters are now encoded as HTML entities, the browser renders <script> as plain visible text rather than parsing it as a tag. The injected payload is displayed harmlessly instead of executing, confirming the vulnerability is fixed.

2. Denial of Service

2.1 The attack performed and proof of attack

The following request was sent using curl: curl -v "http://localhost:8000/%"





The crash originates from serveStatic() in httpd.js, which calls decodeURIComponent(req.url) without any error handling:

var decodedPath = decodeURIComponent(req.url);

If a "%" character is not followed by two valid hexadecimal digits then it is not a well-formed percent-encoded sequence and decodeURIComponent throws a URIError. This exception is never caught so Node considers it to be an uncaught exception which by default kills the whole process and not just the request that went wrong. This means that a single malicious request from any client is enough to take the server down for every other client that is currently connected, which makes it a real denial of service vulnerability.

2.2 The countermeasures taken

The decodeURIComponent(req.url) call in serveStatic() was wrapped in a try/catch block. When decoding fails, the server now responds with 400 Bad Request and a plain-text error message, and returns immediately, instead of allowing the exception to propagate uncaught.

```
var decodedPath;
try {
  decodedPath = decodeURIComponent(req.url);
} catch (err) {
    res.writeHead(400, { "Content-Type": "text/plain" });
    res.end("400 Bad Request: malformed URL encoding");
    return;
  }
```

Re-sending the same request confirms the fix:

curl -v "http://localhost:8000/%"

```
nasif@Nasifs-MacBook-Air Patched % curl -v "http://localhost:8000/%"
* Host localhost:8000 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8000...
* Connected to localhost (::1) port 8000
> GET /% HTTP/1.1
> Host: localhost:8000
> User-Agent: curl/8.7.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 400 Bad Request
< Content-Type: text/plain
< Date: Mon, 14 Sep 2026 16:46:56 GMT
< Connection: keep-alive
< Keep-Alive: timeout=5
< Transfer-Encoding: chunked
<
* Connection #0 to host localhost left intact
400 Bad Request: malformed URL encoding%
```

```
nasif@Nasifs-MacBook-Air Patched % node httpd.js
GET /% 1.1
Content-Type: undefined
Body:
```

The server now handles malformed input gracefully. A single malformed request can no longer take down the entire server; only that one request is rejected, while the server continues serving all other clients normally.